COMP 170 → COMP 271 Diagnostic Check Quiz

Answers and explanations (for review and study).

1. **Variables & Mutability**
What will be printed?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

(A) `[1, 2, 3]`
(B) `[1, 2, 3, 4]`
(C) `[4]`
(D) Error

Answer: (B)
Feedback: *Lists are mutable, and `b = a` makes `b` refer to the same list object as `a`.* Calling `b.append(4)` modifies that shared list, so printing `a` shows `[1, 2, 3, 4]`. If you chose (A), you may be thinking assignment copies the list (it does not). If you chose (C), note that `append` adds to the existing list rather than replacing it.

2. **Function Calls & Assignment**
What is printed?

```
def mystery(x):
    x = x + 1
    return x

y = 5
mystery(y)
print(y)
```

(A) `6`
(B) `5`
(C) `None`
(D) Error

Answer: (B)
Feedback: *`y` is unchanged because the function modifies only its local parameter variable `x`.* Also, integers are immutable, and the returned value is not stored anywhere. If you chose (A), you likely assumed the function "updates" `y`; it does not unless you assign: `y = mystery(y)`.

3. **Lists vs Tuples**
Which statement is **true**?

(A) Lists are immutable, tuples are mutable
(B) Tuples allow item reassignment
(C) Lists allow item reassignment
(D) Lists and tuples behave identically

Answer: (C)
Feedback: *Lists are mutable (you can reassign items), while tuples are immutable (you cannot).* If you chose (A) or (B), the mutability is reversed. If you chose (D), remember that mutability and intended usage differ.

4. **Loop Execution**
How many times does the loop run?

```
for i in range(2, 10, 2):
    print(i)
```

(A) 3
(B) 4
(C) 5
(D) 6

Answer: (B)
Feedback: *`range(2, 10, 2)` produces 2, 4, 6, 8 (stop value 10 is excluded).* That is 4 values $\Rightarrow$ the loop runs 4 times.

5. **Conditionals**
What is printed?

```
x = 7
if x % 2 == 0:
    print("Even")
elif x > 5:
    print("Big")
else:
    print("Odd")
```

(A) `Even`
(B) `Odd`
(C) `Big`
(D) Nothing

Answer: (C)
Feedback: *Since 7 is not even, the first condition is false. The `elif` condition `x > 5` is true, so it prints `Big`.* If you chose (B), you skipped over the `elif` branch; `else` only runs when all earlier conditions fail.

6. **List Indexing**
What does this print?

```
lst = [10, 20, 30, 40]
print(lst[-1])
```

(A) `10`
(B) `20`
(C) `30`
(D) `40`

Answer: (D)
Feedback: *A negative index counts from the end; `-1` refers to the last element.* The last element is 40.

7. **Boolean Logic**
What is the value of the expression?

```
True and not False or False
```

(A) `True`
(B) `False`
(C) Error
(D) Depends on parentheses

Answer: (A)
Feedback: *Operator precedence is: `not` first, then `and`, then `or`.* So `not False` is `True`; then `True and True` is `True`; then `True or False` is `True`.

8. **Dictionaries**
Which operation retrieves a value from dictionary `d` using key `"id"`?

(A) `d("id")`
(B) `d[id]`
(C) `d["id"]`
(D) `d.get[id]`

Answer: (C)
Feedback: *Dictionary lookup uses square brackets with the key: `d["id"]`.* (A) incorrectly calls the dictionary as a function. (B) uses a variable named `id` (not the string key). (D) misuses `get` (it should be `d.get("id")`).

9. **Classes & Objects**
What does `self` represent in a Python class?

(A) The class itself
(B) A static variable
(C) The current object instance
(D) A global reference

Answer: (C)
Feedback: *`self` refers to the specific object (instance) whose method is currently running.* It is how methods access instance variables (e.g., `self.value`).

10. **Method Calls**
Given:

```
class Counter:
    def __init__(self):
        self.value = 0

    def inc(self):
        self.value += 1
```

What does this code print?

```
c = Counter()
c.inc()
print(c.value)
```

(A) `0`
(B) `1`
(C) Error
(D) `None`

Answer: (B)
Feedback: *`inc` increases the instance variable `value` by 1.* After one call, `c.value` becomes 1, so it prints 1. If you chose (D), note that `inc` returns `None`, but we are printing `c.value`, not the method's return.

11. **Variable Scope**
What is printed?

```
x = 10

def f():
    x = 5

f()
print(x)
```

(A) `5`
(B) `10`
(C) Error
(D) `None`

Answer: (B)
Feedback: *The assignment `x = 5` inside `f` creates a new local variable `x`.* It does not change the global `x`, so printing outside the function prints 10.

12. **List Operations**
Which operation adds an element to the **end** of a list?

(A) `lst.add(x)`
(B) `lst.insert(x)`
(C) `lst.append(x)`
(D) `lst.push(x)`

Answer: (C)
Feedback: *`append` adds a single element to the end of a Python list.* `insert` exists, but it requires an index: `lst.insert(i, x)`.

13. **Return vs Print**
Which statement is **correct**?

(A) `print` sends a value back to the caller
(B) `return` displays output on screen
(C) `return` sends a value back to the caller
(D) `print` ends function execution

Answer: (C)
Feedback: *`return` sends a value back to the code that called the function.* `print` displays text but does not return it. Also, `print` does not necessarily end a function; `return` ends the function at that point.

14. **Performance Intuition (Pre-DS)**
Which operation is **generally** fastest for large lists?

(A) Searching for an element
(B) Appending an element
(C) Removing from the middle
(D) Sorting the list

Answer: (B)
Feedback: *Appending is typically fast (often constant-time on average).* Searching can be linear-time, removing from the middle requires shifting elements, and sorting is much more expensive than a single append.

15. **Reading Code**
What does this function compute?

```
def f(lst):
    total = 0
    for x in lst:
        total += x
    return total
```

(A) Maximum value
(B) Average
(C) Sum of elements
(D) Number of elements

Answer: (C)
Feedback: *The loop adds each element to `total`, so the final returned value is the sum.* To compute an average, you would also need to divide by the number of items.